



PyQuest

Information- & Aufgabenblatt

Kapitel 11: Funktionen

Baue eigene Bausteine, die du immer wieder benutzen kannst.

Funktionen definieren und aufrufen

Bisher hast du Code immer **von oben nach unten** ausgeführt. Aber was, wenn du dieselben Anweisungen an **mehreren Stellen** im Programm brauchst? Dafür gibt es **Funktionen**.

Eine Funktion ist ein **Unterprogramm**: ein benannter Codeblock, der eine bestimmte Aufgabe erledigt. Du schreibst ihn **einmal** und kannst ihn danach beliebig oft **aufrufen** – das spart Tipparbeit und macht dein Programm übersichtlicher.

Stell dir ein **Rezept** vor. Du schreibst es **einmal** auf einen Zettel und gibst ihm einen Namen: **Pfannkuchen**. Danach musst du nie wieder alle Schritte aufschreiben – du sagst nur noch „Pfannkuchen!“ und weißt, was zu tun ist.

Genau das ist eine **Funktion**: ein Stück Programm, das du **einmal aufschreibst** und dem du einen **Namen** gibst. Danach rufst du es über diesen Namen auf – so oft du willst.

Dabei passieren zwei ganz verschiedene Dinge, und die darfst du nicht verwechseln:

Definieren heißt: Du schreibst das Rezept auf. Es passiert noch **nichts**.

Aufrufen heißt: Du sagst den Namen. **Jetzt** wird gekocht.

So ist ein Funktionskopf aufgebaut

Die erste Zeile einer Funktion heißt **Funktionskopf**. Sie besteht immer aus denselben vier Teilen – in genau dieser Reihenfolge:

Bauplan

```
def begruessung():  
    print("Hallo!")
```



1. **Das Schlüsselwort def** – Kurz für *define* – definieren. Damit weiß Python: Jetzt kommt eine Funktion. Ohne def geht es nicht.
2. **Der Funktionsname** – Den suchst **du** aus. Er sagt, was die Funktion tut. Alles klein, Wörter mit Unterstrich: `zeige_stern`.
3. **Die runden Klammern** – Sie gehören immer dazu – auch wenn nichts drinsteht. Später kommt hier hinein, was die Funktion braucht.
4. **Der Doppelpunkt** – Er schließt den Kopf ab und kündigt an: Ab der nächsten Zeile kommt, was die Funktion tun soll.
5. **Der Rumpf – eingerückt**

Definieren mit `def`, aufrufen mit Klammern:

Beispiel

```
def begruessung():  
    print("Hallo!")  
  
begruessung()
```

Alles, was **eingerückt** unter dem Kopf steht, gehört zur Funktion – das ist ihr **Rumpf**. Die Einrückung ist keine Schönheitssache: Nur an ihr erkennt Python, wo die Funktion anfängt und wo sie aufhört.

Sobald eine Zeile wieder **ganz links** beginnt, ist die Funktion zu Ende.

Und so rufst du sie auf

Der Aufruf ist viel kürzer – kein `def`, kein Doppelpunkt, keine Einrückung. Nur der Name und die Klammern:

Bauplan

```
begruessung()
```



1. **Derselbe Name** – Genau so geschrieben wie oben bei def. Ein Tippfehler, und Python findet die Funktion nicht.

2. **Die Klammern** – Sie sind das eigentliche Startsignal. **Ohne Klammern passiert nichts** – dann meinst du nur den Namen, rufst die Funktion aber nicht auf.

Der Code innerhalb der Funktion (eingerückt) wird erst ausgeführt, wenn die Funktion **aufgerufen** wird – also wenn ihr Name mit Klammern dahinter geschrieben wird: `begrueessung()`.

Merke (PEP 8): Funktionsnamen beginnen in Python mit einem **Kleinbuchstaben** und werden in **snake_case** geschrieben (Wörter durch Unterstriche getrennt), z. B. `produkte_information`.

Aufgabe 1: Mit welchem Schlüsselwort definiert man eine Funktion in Python?

- ☐ a) function
- ☐ b) def
- ☐ c) func

Aufgabe 2: Definiere eine Funktion `zeige_stern`, die einen Stern (*) ausgibt. Rufe sie danach dreimal auf.

Dein Code:

Eine Funktion kann auch **mehrere Zeilen** ausführen – ihr Codeblock funktioniert wie jeder andere eingerückte Block (z. B. bei `if` oder `for`).

Aufgabe 3: Definiere eine Funktion `begrueessen`, die genau zwei Zeilen ausgibt: `Willkommen!` und `Schön, dass du da bist..` Rufe die Funktion zweimal auf.

Dein Code:



Parameter

Mit **Parametern** kannst du einer Funktion Werte übergeben, mit denen sie arbeitet. Die Parameter stehen in den Klammern der Definition.

name ist hier ein Parameter:

Beispiel

```
def begruesse(name):  
    print("Hallo " + name + "!")  
  
begruesse("Ada")  
begruesse("Alan")
```

Bei jedem Aufruf kann ein anderer Wert übergeben werden – die Funktion muss nur einmal geschrieben werden. Du kannst auch **mehrere Parameter** angeben, getrennt durch Kommas: def addiere(a, b):

Aufgabe 4: Was ist der Vorteil von Parametern?

- ☐ a) Die Funktion kann mit unterschiedlichen Werten wiederverwendet werden
- ☐ b) Parameter machen den Code langsamer
- ☐ c) Parameter sind nur für Zahlen erlaubt

Aufgabe 5: Definiere eine Funktion addiere(a, b), die die Summe von a und b ausgibt. Rufe sie auf mit addiere(3, 4) und addiere(10, 5).

Dein Code:

Bei **mehreren** Parametern zählt die **Reihenfolge**: Der erste übergebene Wert landet im ersten Parameter, der zweite im zweiten, und so weiter.

```
def buch_info(titel, autor):  
    print(titel, "von", autor)  
  
buch_info("1984", "George Orwell")
```



Aufgabe 6: Definiere eine Funktion `buch_info(titel, autor, preis)`, die die drei Werte im Format `Titel: ..., Autor: ..., Preis: ... Euro` ausgibt.

Rufe sie auf mit `buch_info("Moby Dick", "Herman Melville", 14.99)`.

Dein Code:

Standardwerte für Parameter

Manchmal soll ein Parameter **optional** sein – die Funktion soll auch **ohne** ihn funktionieren. Dafür gibst du dem Parameter einen **Standardwert** (Default): `parameter=wert`.

Wird beim Aufruf **kein** Wert für diesen Parameter übergeben, wird automatisch der Standardwert benutzt.

`rabatt` hat den Standardwert 0 – wird er nicht angegeben, gibt es keinen Rabatt:

Beispiel

```
def preis_anzeigen(preis, rabatt=0):  
    print(preis - rabatt)  
  
preis_anzeigen(100)  
preis_anzeigen(100, 20)
```



In Python gibt es (anders als z. B. in Java) **kein** „Überladen“ von Funktionen mit gleichem Namen. Stattdessen macht man Parameter mit Standardwerten **optional** – so kann eine einzige Funktion flexibel auf verschiedene Aufrufe reagieren.

Wichtig: Parameter **mit** Standardwert müssen in der Definition immer **nach** den Parametern **ohne** Standardwert stehen.

Aufgabe 7: Was passiert bei `def gruss(name, text="Hallo"):`, wenn du die Funktion nur mit `gruss("Ada")` aufrufst?

- ☐ a) Es gibt einen Fehler, weil text fehlt
- ☐ b) text bekommt automatisch den Wert "Hallo"
- ☐ c) text bleibt leer

Aufgabe 8: Definiere eine Funktion `kunden_info(name, alter, buch=None)`. Ist buch None, gib `Name: ..., Alter: ...` aus. Sonst gib zusätzlich das Buch aus: `Name: ..., Alter: ..., Gekauftes Buch: ....`

Rufe die Funktion zweimal auf: `kunden_info("Anna", 28)` und `kunden_info("Max", 30, "Harry Potter")`.

Dein Code:

Rückgabewerte

Mit **return** kann eine Funktion ein Ergebnis **zurückgeben**, das du danach weiterverwenden kannst – zum Beispiel in einer Variable speichern.

Das ist etwas anderes als `print()`: `print()` zeigt nur etwas an, `return` gibt einen Wert an die Stelle zurück, wo die Funktion aufgerufen wurde.

`quadrat` gibt einen Wert zurück, den wir speichern:

Beispiel

```
def quadrat(zahl):  
    return zahl * zahl
```



```
ergebnis = quadrat(5)
print(ergebnis)
```

Sobald Python auf `return` trifft, verlässt es die Funktion sofort mit diesem Wert. Hat eine Funktion **kein** `return`, gibt sie automatisch **None** zurück ("nichts").

Aufgabe 9: Was gibt eine Funktion zurück, die kein `return` enthält?

- ☐ a) 0
- ☐ b) None
- ☐ c) Einen Fehler

Aufgabe 10: Definiere eine Funktion `verdopple(zahl)`, die mit `return` (nicht mit `print`!) das Doppelte der Zahl zurückgibt.

Dein Code:



Ein zurückgegebener Wert kann sofort **weitereverwendet** werden – zum Beispiel als Argument für eine andere Berechnung, oder direkt in `print()`. So kombinierst du mehrere Funktionen miteinander.

Aufgabe 11: Definiere eine Funktion `durchschnitt(a, b, c)`, die mit `return` den Durchschnitt der drei Zahlen zurückgibt.

Rufe sie mit `durchschnitt(10.5, 15.0, 9.0)` auf, speichere das Ergebnis in einer Variable und gib es aus.

Erwartet: **11.5**

Dein Code:

Globale Variablen verändern

Eine Variable, die **außerhalb** aller Funktionen angelegt wird, heißt **globale Variable**. Du kannst sie in einer Funktion zwar **lesen** – aber wenn du ihr innerhalb der Funktion einen **neuen Wert zuweisen** willst, brauchst du das Schlüsselwort **global**.

Ohne `global` würde Python eine NEUE lokale Variable anlegen, statt die globale zu verändern:

Beispiel

```
anzahl = 0

def erhoehen():
    global anzahl
    anzahl += 1

erhoehen()
erhoehen()
print(anzahl)
```




`global anzahl` sagt Python: „Ich meine hier **die** globale Variable `anzahl`, nicht eine neue lokale.“ Ohne diese Zeile würde `anzahl += 1` einen Fehler auslösen, weil Python `anzahl` sonst als **neue, lokale** Variable behandelt, die beim Lesen noch keinen Wert hat.

Typischer Einsatz: ein Zähler oder eine Summe, die über mehrere Funktionsaufrufe hinweg **erhalten bleiben** soll – zum Beispiel ein Lagerbestand, der bei jedem Verkauf sinkt.

Aufgabe 12: Wofür brauchst du `global` in einer Funktion?

- ☐ a) Um eine globale Variable zu lesen
- ☐ b) Um einer globalen Variable innerhalb der Funktion einen neuen Wert zuzuweisen
- ☐ c) Um eine Funktion global aufrufbar zu machen

Aufgabe 13: Lege die globale Variable `lagerbestand = 24` an. Definiere eine Funktion `verkaufen()`, die den Lagerbestand um 1 verringert (mit `global`) und ihn danach ausgibt. Rufe `verkaufen()` dreimal auf.

Erwartet: 23, 22, 21 (untereinander).

lagerbestand = 24

Dein Code:

Funktionen anwenden

Jetzt kombinierst du alles: **Parameter**, **return** und **global**. Ein typischer Einsatz ist ein **Zähler**, der bei jedem Funktionsaufruf um eins steigt und danach zurückgegeben wird – zum Beispiel, um Kunden in einem Laden zu zählen.

Jeder Aufruf begrüßt eine Person UND zählt sie mit:

Beispiel

```
kunden_zahl = 0
```

```
def begruessen_und_zaehlen(name):  
    global kunden_zahl
```



```
kunden_zahl += 1
print("Hallo", name)
return kunden_zahl

print(begruessen_und_zaehlen("Ada"))
print(begruessen_und_zaehlen("Alan"))
```

Aufgabe 14: Lege die globale Variable `gesamtsumme = 0.0` an. Definiere eine Funktion `kassieren(betrag)`, die `betrag` zur `gesamtsumme` addiert (mit `global`) und die neue **Gesamtsumme** zurückgibt.

Rufe `kassieren` dreimal auf – mit 12.5, 8.0 und 20.0 – und gib bei jedem Aufruf das **Ergebnis** aus (`print(kassieren(...))`).

Erwartet: 12.5, 20.5, 40.5 (untereinander).

gesamtsumme = 0.0

Dein Code:

Eine Funktion kann auch **je nach Situation** eine andere Meldung zurückgeben – zum Beispiel, wenn ein Platzlimit erreicht ist. Das kombiniert `if/else`, `global` und `return` in einer Funktion.

Aufgabe 15: Lege die globale Variable `anzahl_teilnehmer = 0` an. Definiere eine Funktion `lesung_anmelden()` (ohne Parameter): Ist `anzahl_teilnehmer` kleiner als 3, erhöhe sie um 1 (mit `global`) und gib **Angemeldet!** zurück. Ist das Limit erreicht, gib **Warteliste!** zurück.

Rufe `lesung_anmelden()` **viermal** auf und gib jedes Ergebnis aus.

Erwartet: Angemeldet!, Angemeldet!, Angemeldet!, Warteliste! (untereinander).

anzahl_teilnehmer = 0

Dein Code:



Aufgabe 16: Wiederholung aus Kapitel 4: Schreibe eine Funktion `initialen(vorname, nachname)`, die die Initialen zurückgibt – jeweils der erste Buchstabe groß, mit Punkt dahinter.

Beispiel: `initialen("mia", "berger")` ergibt **M.B.**

Rufe die Funktion danach zweimal auf: mit "mia" / "berger" und mit "Tom" / "klein", und gib beide Ergebnisse aus.

```
def initialen(vorname, nachname):  
    # Hier deine Lösung  
    pass
```

Rufe die Funktion zweimal auf

Dein Code: